Bash Scripts For Automation

1. Print Numbers from 1 to 100

Description

Print numbers from 1 to 100 using a for loop.

```
#!/bin/bash

for i in {1..100}
do
    echo "$i"
done
```

Explanation

```
#!/bin/bash
```

Specifies the Bash interpreter.

```
for i in {1..100}
```

Creates a loop from 1 to 100.

```
do
```

Beginning of loop body.

```
echo "$i"
```

Prints current value.

```
done
```

Ends the loop.

---

2. Find Largest Among Three Numbers

Description

Compare three numbers and print the largest.

```
#!/bin/bash

a=20
b=50
c=30

largest=$a

if [ "$b" -gt "$largest" ]; then
    largest=$b
fi

if [ "$c" -gt "$largest" ]; then
    largest=$c
fi

echo "Largest number is $largest"
```

Explanation

```
largest=$a
```

Assume first number is largest.

```
if [ "$b" -gt "$largest" ]
```

Checks if b is greater.

```
largest=$b
```

Updates largest.

Same logic for c.

---

3. Check Whether File Exists

Description

Checks whether a file exists.

```
#!/bin/bash

FILE="/tmp/test.txt"

if [ -f "$FILE" ]; then
    echo "File exists"
else
    echo "File does not exist"
fi
```

Explanation

`-f`
Checks whether the path exists and is a regular file.

---

4. Count Lines in File

Description

Counts total number of lines.

```
#!/bin/bash

FILE="file.txt"

count=$(wc -l < "$FILE")

echo "Total lines: $count"
```

Explanation

`wc -l`

Counts lines.

`<`

Redirects file into wc.

---

5. Find Top 5 Largest Files

Your script works but is slower.

Better version:

```
#!/bin/bash

find / -type f -exec ls -lhS {} + 2>/dev/null | head -5
```

or

```
find / -type f -printf "%s %p\n" 2>/dev/null | sort -nr | head -5
```

Description

Finds five largest files.

Explanation

`find /`

Searches filesystem.

`-type f`

Only files.

`sort -nr`

Sort numerically in reverse.

`head -5`

Displays first five.

---

6. Check if Service is Running

```
#!/bin/bash

service="nginx"

if systemctl is-active --quiet "$service"
then
    echo "$service is running"
else
    echo "$service is stopped"
fi
```

Explanation

`systemctl is-active`

Checks service state.

`--quiet`

Suppresses output.

Exit code determines result.

---

7. Monitor Disk Usage

```
#!/bin/bash

usage=$(df / | awk 'NR==2 {gsub("%",""); print $5}')
```

```
if [ "$usage" -gt 80 ]
then
    echo "ALERT: Disk usage is ${usage}%"
else
    echo "Disk usage is normal (${usage}%)"
fi
```

Explanation

```
df /
```

Displays filesystem usage.

```
awk
```

Extracts fifth column.

```
gsub("%","")
```

Removes % symbol.

---

8. Check CPU Utilization

Your script only prints one CPU field.

Better version:

```
#!/bin/bash

cpu=$(top -bn1 | grep "Cpu(s)" | awk '{print 100-$8}')

echo "CPU Usage: ${cpu}%"
```

or

```
mpstat 1 1 | awk '/Average/ {print 100-$NF "%"}'
```

Explanation

Idle CPU is column 8.

Usage =

```
100 - idle
```

---

9. Find Duplicate Entries

```
#!/bin/bash

sort file.txt | uniq -d
```

Explanation

Sorts file first because `uniq` only detects adjacent duplicates.

---

10. Reverse String

```
#!/bin/bash

str="devops"

echo "$str" | rev
```

Explanation

`rev` reverses each line.

---

11. Count Occurrence of Word

```
#!/bin/bash

grep -c "ERROR" app.log
```

Explanation

`-c`

Counts matching lines.

---

12. Print Even Numbers

Can be simplified.

```
#!/bin/bash

for ((i=2;i<=100;i+=2))
do
    echo "$i"
done
```

This is more efficient.

---

13. Extract Failed SSH Logins

```
grep "Failed password" /var/log/auth.log
```

For RHEL

```
grep "Failed password" /var/log/secure
```

Explanation

Searches failed SSH login attempts.

---

14. Parse CSV File

```
#!/bin/bash

while IFS=',' read -r id name tests
do
    echo "ID=$id Name=$name Tests=$tests"
done < students.csv
```

Explanation

```
IFS=','
```

Uses comma as delimiter.

```
read -r
```

Prevents backslash interpretation.

---

15. Student with Least Tests

Works perfectly.

Only add:

```
read -r
```

if using shell.

The awk logic is excellent.

---

16. Backup Directory

```
#!/bin/bash

SOURCE="/var/www"
DEST="/backup"

mkdir -p "$DEST"

tar -czf "$DEST/backup_$(date +%F).tar.gz" "$SOURCE"

echo "Backup completed"
```

Added

```
mkdir -p
```

So backup directory always exists.

---

17. Delete Old Logs

```
find /var/log -type f -mtime +30 -delete
```

Explanation

```
mtime
```

Last modified days.

```
+30
```

Older than 30 days.

---

18. Website Availability

Instead of checking only 200:

```
#!/bin/bash

URL="https://google.com"

status=$(curl -o /dev/null -s -w "%{http_code}" "$URL")

if [[ "$status" =~ ^2|3 ]]
then
```

```
    echo "Website is UP"
else
    echo "Website is DOWN"
fi
```

Because redirects (301/302) are also healthy.

---

19. Kubernetes Pod Health

Your script prints non-running pods.

Interviewers usually expect:

```
#!/bin/bash

kubectl get pods --no-headers | awk '$3!="Running"{print}'
```

Explanation

Column 3

```
STATUS
```

Shows Pending

CrashLoopBackOff

ImagePullBackOff

etc.

---

20. Count Log Levels

Status: ✅ Excellent

```
#!/bin/bash

awk '
{
    count[$1]++
}
END {
    for(i in count)
        print i, count[i]
}' app.log
```

Explanation

Associative array

```
count["INFO"]
count["ERROR"]
count["WARN"]
```

Counts occurrences automatically.